

Thực hành phát triển hệ thống Trí tuệ nhân tạo Phân tích thiết kế hướng đối tượng (OOAD)

Trần Quốc Long

Trường ĐH Công nghệ, DHQG Hà Nội

Ngày 1 tháng 3 năm 2026

Mục lục

1 Lập trình hướng đối tượng

2 Nguyên tắc SOLID

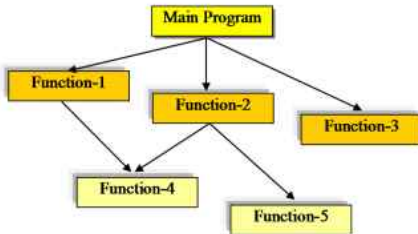
3 Một số nguyên tắc khác

Các nguyên tắc trong lập trình hướng đối tượng

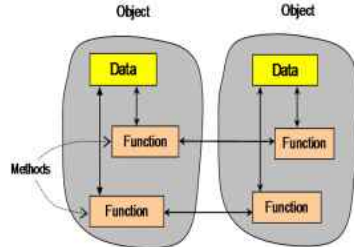
- *Trừu tượng* (Abstraction)
- *Đóng gói* (Encapsulation)
- *Mô-đun* (Modularity)
- *Phân cấp* (Hierarchy)
- *Đa hình* (Polymorphism)
- *Thừa kế* (Inheritance)
- *Hợp thành* (Composition)
- *Giao tiếp* (Delegation)
- *Phân loại* (Subtyping)
- *Ẩn dữ liệu* (Data Hiding)
- *Tách biệt quan tâm*
(Separation of Concerns)

Trừu tượng hoá

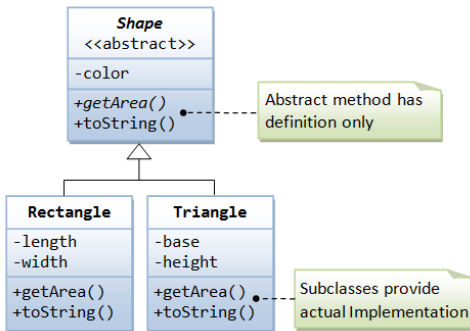
Procedure-oriented Programming



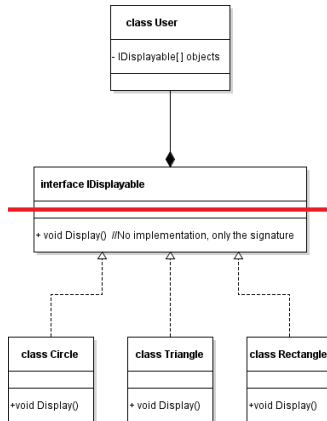
Object-oriented Programming



Trừu tượng và đa hình

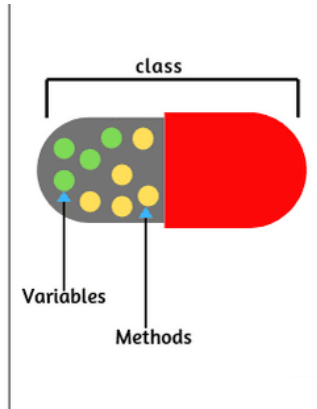


Trừu tượng và đa hình



Đóng gói

```
class
{
    data members
    +
    methods (behavior)
}
```



Thực hành phát triển hệ thống Trí tuệ nhân tạo Phân tích thiết kế hướng đối tượng (OOAD)

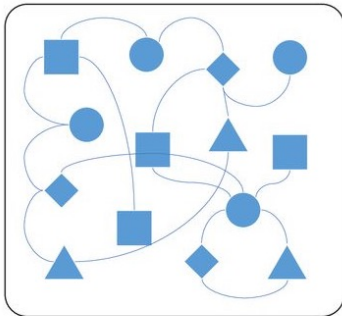


Đóng gói và Ẩn dữ liệu²

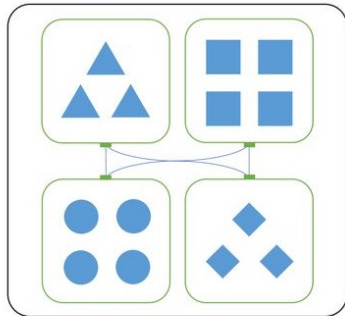
```
class Temperature {  
    private int t;  
    public int getT() { return this.t; }  
    public void setT(int t) { this.t = t; }  
}
```

```
class Temperature {  
    private int t;  
    public String toString() {  
        return String.format("%d F", this.t);  
    }  
}
```

Mô-đun

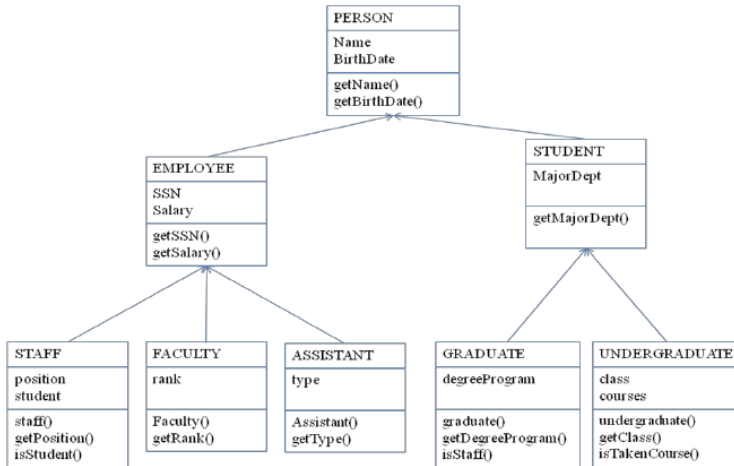


Non-modular software

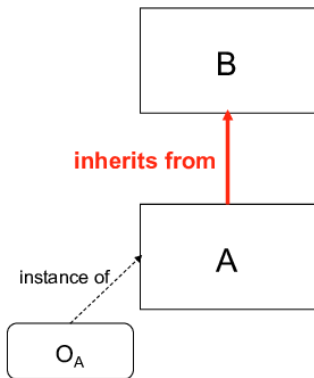


Modular software

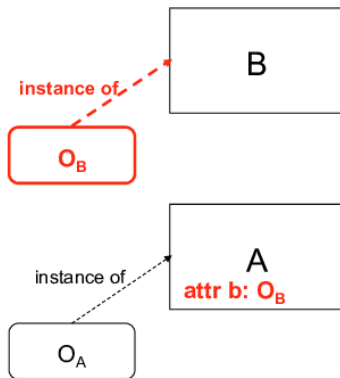
Phân cấp



Thừa kế và Hợp thành

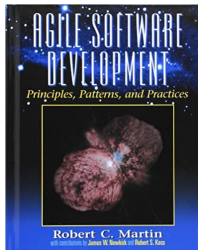


Inheritance



Composition

Các nguyên tắc SOLID



Trong Agile, thái độ của nhà phát triển đối với thiết kế phần mềm giống như thái độ của các bác sĩ phẫu thuật đối với quy trình vô trùng. Quy trình vô trùng điều kiện tiên quyết của phẫu thuật. Nếu không có nó, nguy cơ nhiễm trùng sẽ quá cao để chịu đựng. Các nhà phát triển Agile cũng cảm thấy tương tự đối với thiết kế của họ.

Robert Martin, *Agile Software Development. Principles, Patterns, and Practices*

Các nguyên tắc SOLID

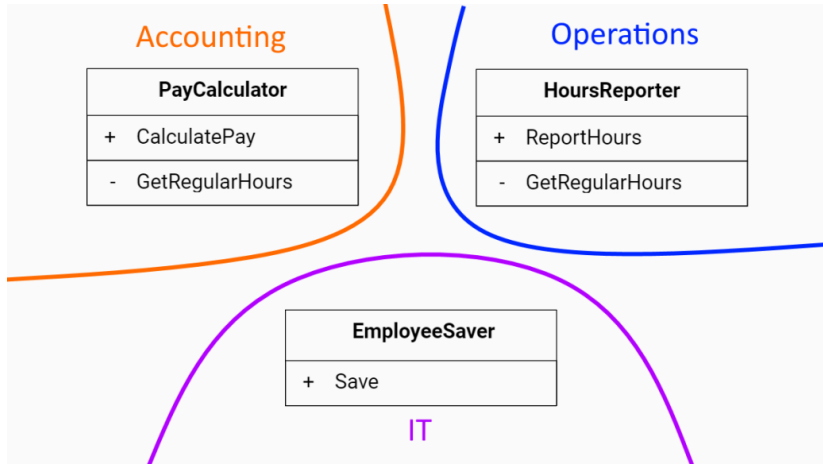
- **S**ingle Responsibility Principle - đơn trách nhiệm
- **O**pen Close Principle - mở đóng
- **L**iskov Substitution Principle - thay thế Liskov
- **I**nterface Segregation Principle - phân chia giao diện
- **D**ependency Inversion Principle - đảo ngược phụ thuộc

Nguyên tắc đơn trách nhiệm

- Một lớp/mô-đun chỉ nên có một lý do để thay đổi
- Mỗi lớp/mô-đun chỉ nên thực hiện một chức năng cụ thể và phải đóng gói chức năng đó
- Mỗi lớp/mô-đun chỉ nên thực hiện một vai trò

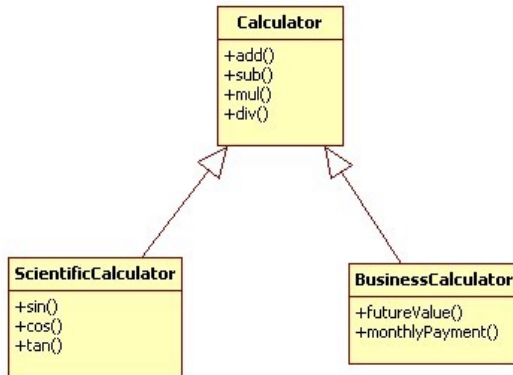
Nên dùng	Không nên dùng
■ Lớp có nhiều chức năng không liên quan ■ Lớp quá lớn ■ Lớp có bộ phận hay thay đổi	■ Phân tách lớp khiến code phức tạp hơn ■ Phạm vi của lớp nhỏ, các thành phần liên quan mạnh ■ Hiệu suất giảm quá nhiều

Nguyên tắc đơn trách nhiệm



Nguyên tắc mở đóng

Một lớp phải **mở** để mở rộng nhưng **đóng** đối với sửa đổi



Nguyên tắc thay thế Liskov

Nếu với mỗi đối tượng o_1 thuộc lớp S có đối tượng o_2 thuộc lớp T sao cho mọi chương trình P viết dựa trên T mà hành vi của P không thay đổi khi o_1 thay thế o_2 thì S là lớp con của T –

Barbara Liskov

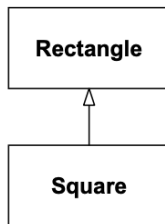


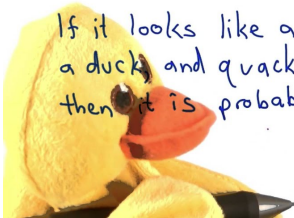
Figure 10-1 Square inherits from Rectangle

Duck Typing

Nếu nó trông như vịt, bơi như vịt và kêu như vịt, thì chắc nó là vịt

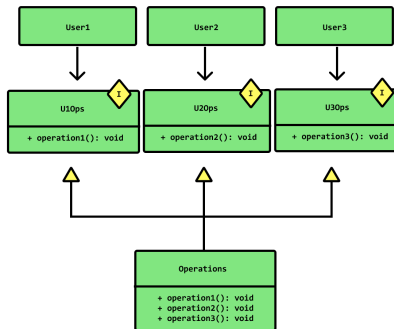
"Duck Typing"

If it looks like a duck, swims like a duck, and quacks like a duck, then it is probably a duck!



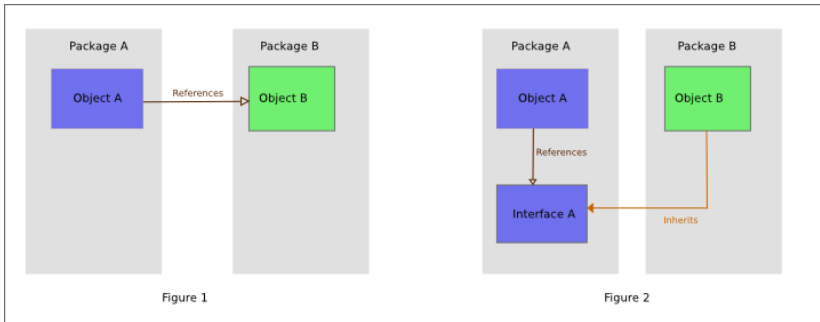
Nguyên tắc phân chia giao diện

Đối tượng khách (client) không nên bị ép buộc phụ thuộc vào các phương thức mà nó không sử dụng – **Robert Martin**

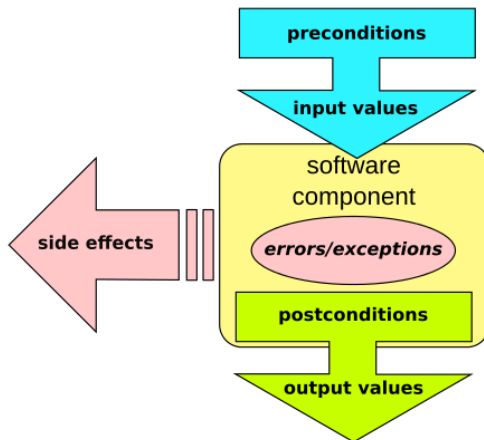


Nguyên tắc đảo ngược phụ thuộc

(a) Các module cấp cao không nên phụ thuộc vào các module cấp thấp. Cả hai nên phụ thuộc vào các giao diện (b) Các giao diện không nên phụ thuộc vào chi tiết. Chi tiết nên phụ thuộc vào giao diện

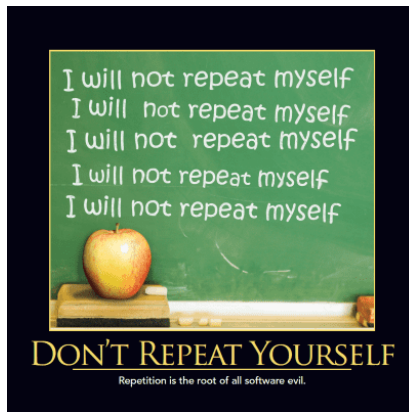


Thiết kế dựa trên hợp đồng



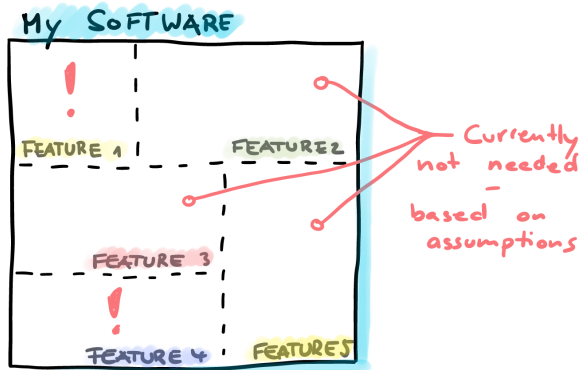
Tránh trùng lặp

Don't Repeat Yourself (DRY)



Giảm lược tính năng

You Ain't Gonna Need It (YAGNI)



Đảo ngược kiểm soát

Inversion of Control (IoC)

Look at this code:

```
| print(book.title());
```

It is very straight forward: we retrieve the title from the book and simply give it to the `print()` procedure, or whatever else it might be. *We* are in charge, the *control* is in our hands.

In contrast to this, here is the *inversion*:

```
| print(book);
```

We give the entire book to the procedure `print()` and it calls `title()` when it feels like it. That is, we *delegate* control.

OOP và Lập trình hàm

OO pattern/principle

- Single Responsibility Principle
- Open/Closed principle
- Dependency Inversion Principle
- Interface Segregation Principle
- Factory pattern
- Strategy pattern
- Decorator pattern
- Visitor pattern

FP pattern/principle

- Functions
- Functions
- Functions, also
- Functions
- Yes, functions
- Oh my, functions again!
- Functions
- Functions ☐

Nhược điểm của OOP



Edsger W. Dijkstra (1989)

"TUG LINES," Issue 32, August 1989

"Object oriented programs are offered as alternatives to correct ones" and "Object-oriented programming is an exceptionally bad idea which could only have originated in California."

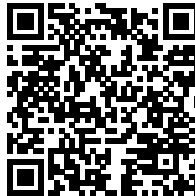


Alan Kay (1997)

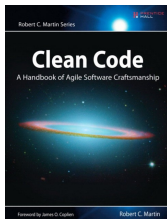
The Computer Revolution hasn't happened yet

"I invented the term object-oriented, and I can tell you I did not have C++ in mind." and "Java and C++ make you think that the new ideas are like the old ones. Java is the most distressing thing to happen to computing since MS-DOS." ([proof](#))

What's Wrong With Object-Oriented Programming? (2016)



Tài liệu tham khảo

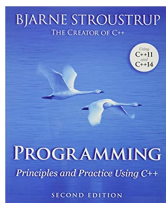


Hình: Robert C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*

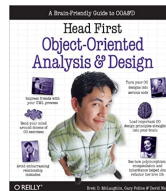


Hình: Grady Booch và cộng sự, *Object-Oriented Analysis and Design with Applications*

Tài liệu tham khảo



Hình: Bjarne Stroustrup,
Programming: Principles and Practice Using C++



Hình: Brett McLaughlin và cộng sự,
Head First Object-Oriented Analysis and Design: A Brain Friendly Guide to OOA&D